



خلاصه

این آزمایش سه بخش دارد: پیمانه‌ی جمع و تفریق اعداد مختلط، پیمانه‌ی ضرب اعداد مختلط، و یک واحد پایپ‌لاین که دستورات را از یک حافظه‌ی ۳۲ کلمه‌ای می‌خواند و عملیات مختلط را روی آن‌ها انجام می‌دهد.

قید کوتاهی در انتهای صورت آزمایش آمده که در عمل تمام معماری را تعیین می‌کند:

«در این آزمایش، نایستی از واحدهای جمع کننده یا ضرب کننده بیش از یک بار استفاده شود.»

یعنی در کل ماشین یک جمع‌کننده و یک ضرب‌کننده وجود دارد. تمام تصمیم‌هایی که در ادامه می‌آید — یکی بودن ماژول جمع/تفریق و ضرب، زمان‌بندی چهار مرحله‌ای ضرب، و اینکه مرحله‌ی اجرای پایپ‌لاین چند کلاکی است — پیامد مستقیم همین یک جمله است.

نمایش عدد مختلط

هر عدد مختلط یک زوج از دو مؤلفه‌ی علامت‌دار مکمل ۲ هشت‌بیتی است که در یک کلمه‌ی ۱۶ بیتی بسته‌بندی می‌شود:

$$\text{word} = \{ \text{imag}[7:0], \text{real}[7:0] \} \quad \text{بازه‌ی هر مؤلفه: } ۱۲۸ - \text{تا } ۱۲۷ +$$

نتایج اشباع می‌شوند نه سرریز: مؤلفه‌ای که از بازه بیرون بزند به ۱۲۸- یا ۱۲۷+ چسبانده می‌شود و پرچمی بالا می‌رود. برای ماشینی که کارش حساب روی کمیت‌های اندازه‌گیری‌شده است این رفتار مفیدتر است — مقداری که خیلی بزرگ شده باید «خیلی بزرگ» خوانده شود، نه اینکه بی‌سروصدا علامتش عوض شود. مثلاً $۱۰۰۰ = ۱۰۰ \times ۱۰۰$ با سرریز عادی به ۱۶ تبدیل می‌شد که هیچ ربطی به پاسخ درست ندارد.

چرا جمع/تفریق و ضرب یک ماژول‌اند

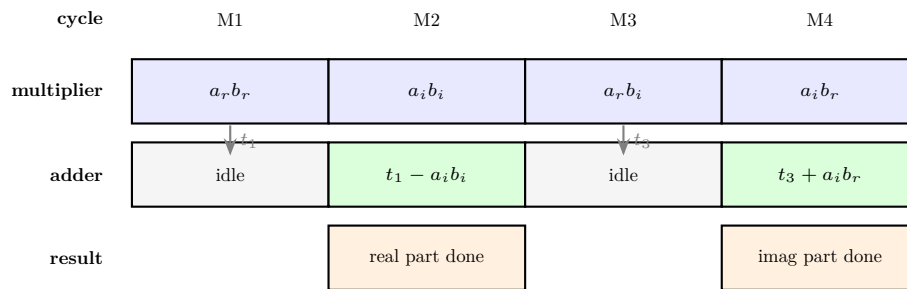
ضرب دو عدد مختلط به چهار ضرب حقیقی و دو جمع حقیقی نیاز دارد:

$$(a_r + j a_i)(b_r + j b_i) = (a_r b_r - a_i b_i) + j (a_r b_i + a_i b_r)$$

و جمع دو عدد مختلط به دو جمع حقیقی دیگر. اگر بخش الف و بخش ب دو ماژول جدا می‌بودند، سطح بالای ماشین دو جمع‌کننده می‌داشت — دقیقاً همان چیزی که قید بالا ممنوع کرده است.

بنابراین هر دو عملیات در یک ماژول به نام `complex_alu` زندگی می‌کنند که داخل آن دقیقاً یک `mul_unit` و یک `add_unit` قرار دارد. بخش الف و بخش ب دو حالت کاری این ماژول‌اند و هر کدام تست‌بنچ مستقل خودشان را دارند. بهایی که برای این قید پرداخت می‌شود در زمان است نه در سطح: ماژول به‌جای ترکیبی، ترتیبی است.

زمان‌بندی واحد مشترک



نکته‌ی اصلی این زمان‌بندی در ستون‌های M2 و M4 است: جمع‌کننده روی حاصل‌ضربی کار می‌کند که در همان کلاک از ضرب‌کننده بیرون می‌آید، در حالی که ضرب‌کننده هم بی‌کار نمانده و مشغول حاصل‌ضرب بعدی است. بدون این هم‌پوشانی، ضرب مختلط ۴ کلاک ضرب به علاوه‌ی ۲ کلاک جمع، یعنی ۶ کلاک، طول می‌کشید. با آن، ۴ کلاک — و هیچ واحد اضافه‌ای هم به مدار اضافه نشده است.

عملیات	کلاک اشغال واحد مشترک	علت
ADD / SUB	۲	دو جمع حقیقی، یک جمع‌کننده
MUL	۴	چهار ضرب حقیقی، یک ضرب‌کننده

هر دو عدد در تست‌بنچ‌ها صریحا بررسی شده‌اند، نه فقط نتیجه‌ی عددی؛ چون درست بودن پاسخ به‌تنهایی نشان نمی‌دهد که مدار واقعا از یک واحد استفاده کرده باشد.

```

1  `include "complex_pkg.vh"
2
3  module complex_alu (
4      input wire      Clk,
5      input wire      RstN,
6      input wire      start,
7      input wire [1:0] op,      // ALU_ADD / ALU_SUB / ALU_MUL
8      input wire [CX_WORD-1:0] a,  // { imag, real }
9      input wire [CX_WORD-1:0] b,
10     output reg [CX_WORD-1:0] y,
11     output reg      ovf,      // a component had to be clamped
12     output wire     busy,
13     output reg      done      // 1 clk pulse, y valid in that cycle
14 );
15     localparam [2:0] IDLE = 3'd0,
16                    A_RE = 3'd1, A_IM = 3'd2,
17                    M1 = 3'd3, M2 = 3'd4, M3 = 3'd5, M4 = 3'd6;
18
19     reg [2:0]      st;
20     reg [CX_WORD-1:0] a_r, b_r;  // operands latched at start
21     reg [1:0]      op_r;
22     reg signed [15:0] t1, t3;  // latched partial products
23     reg            sat_r;      // real part had to be clamped
24
25     wire signed [7:0] ar = a_r[7:0], ai = a_r[15:8];
26     wire signed [7:0] br = b_r[7:0], bi = b_r[15:8];
27
28     // ---- the one multiplier and the one adder ----
29     reg signed [7:0] mx, my;
30     wire signed [15:0] mp;
31     mul_unit u_mul (.x(mx), .y(my), .p(mp));
32
33     reg signed [16:0] ax, ay;
34     reg            asub;
35     wire signed [16:0] as;
36     add_unit u_add (.x(ax), .y(ay), .sub(asub), .s(as));
37
38     assign busy = (st != IDLE);
39
40     // Clamp a 17-bit result into a signed 8-bit component.
41     function signed [7:0] sat8;
42         input signed [16:0] v;
43         begin
44             if (v > 17'sd127) sat8 = 8'sd127;

```

```

45         else if (v < -17'sd128) sat8 = -8'sd128;
46         else sat8 = v[7:0];
47     end
48 endfunction
49
50 function clipped;
51     input signed [16:0] v;
52     begin
53         clipped = (v > 17'sd127) || (v < -17'sd128);
54     end
55 endfunction
56
57 // ---- operand routing: which numbers reach the two shared units ----
58 always @(*) begin
59     mx = 8'sd0;
60     my = 8'sd0;
61     ax = 17'sd0;
62     ay = 17'sd0;
63     asub = 1'b0;
64     case (st)
65         A_RE: begin
66             ax = {{9{ar[7]}}, ar};
67             ay = {{9{br[7]}}, br};
68             asub = (op_r == `ALU_SUB);
69         end
70         A_IM: begin
71             ax = {{9{ai[7]}}, ai};
72             ay = {{9{bi[7]}}, bi};
73             asub = (op_r == `ALU_SUB);
74         end
75         M1: begin mx = ar; my = br; end
76         M2: begin
77             mx = ai; my = bi; // computing ai*bi ...
78             ax = {t1[15], t1}; // ... while the adder does
79             ay = {mp[15], mp}; // ar*br - ai*bi
80             asub = 1'b1;
81         end
82         M3: begin mx = ar; my = bi; end
83         M4: begin
84             mx = ai; my = br; // computing ai*br ...
85             ax = {t3[15], t3}; // ... while the adder does
86             ay = {mp[15], mp}; // ar*bi + ai*br
87             asub = 1'b0;
88         end
89         default: ;
90     endcase
91 end
92
93 always @(posedge Clk or negedge RstN) begin
94     if (!RstN) begin
95         st <= IDLE;
96         y <= {`CX_WORD{1'b0}};
97         ovf <= 1'b0;
98         done <= 1'b0;
99         a_r <= {`CX_WORD{1'b0}};
100        b_r <= {`CX_WORD{1'b0}};
101        op_r <= 2'd0;
102        t1 <= 16'sd0;
103        t3 <= 16'sd0;
104        sat_r <= 1'b0;
105    end else begin
106        done <= 1'b0;
107        case (st)
108            IDLE: if (start) begin
109                a_r <= a;
110                b_r <= b;
111                op_r <= op;
112                sat_r <= 1'b0;
113                st <= (op == `ALU_MUL) ? M1 : A_RE;
114            end
115
116            A_RE: begin
117                y[7:0] <= sat8(as);
118                sat_r <= clipped(as);
119                st <= A_IM;
120            end
121
122            A_IM: begin
123                y[15:8] <= sat8(as);
124                ovf <= sat_r | clipped(as);
125                done <= 1'b1;
126                st <= IDLE;
127            end
128
129            M1: begin t1 <= mp; st <= M2; end
130            M2: begin
131                y[7:0] <= sat8(as);
132                sat_r <= clipped(as);

```

```

132         st      <= M3;
133     end
134     M3: begin t3 <= mp; st <= M4; end
135     M4: begin
136         y[15:8] <= sat8(as);
137         ovf     <= sat_r | clipped(as);
138         done    <= 1'b1;
139         st      <= IDLE;
140     end
141
142     default: st <= IDLE;
143 endcase
144 end
145 end
146 endmodule

```

دو واحد مشترک خودشان کوچک‌اند. توجه کنید که تفریق واحد دومی نیست: $x - y$ همان جمع‌کننده با معکوس کردن y و یک رقم نقلی ورودی است، که دقیقاً همان چیزی است که سنترکننده می‌سازد.

```

1 module mul_unit (
2     input wire signed [7:0] x,
3     input wire signed [7:0] y,
4     output wire signed [15:0] p
5 );
6     assign p = x * y;
7 endmodule

```

```

1 module add_unit (
2     input wire signed [16:0] x,
3     input wire signed [16:0] y,
4     input wire signed [16:0] sub,
5     output wire signed [16:0] s
6 );
7     assign s = sub ? (x - y) : (x + y);
8 endmodule

```

پهنای جمع‌کننده ۱۷ بیت است، چون همین یک واحد باید دو کار خیلی متفاوت را انجام دهد: جمع دو مؤلفه‌ی ۸ بیتی (که ۹ بیت کافی بود) و ترکیب دو حاصل ضرب جزئی ۱۶ بیتی (که ۱۷ بیت لازم دارد).

ماشین: مجموعه دستورات و حافظه

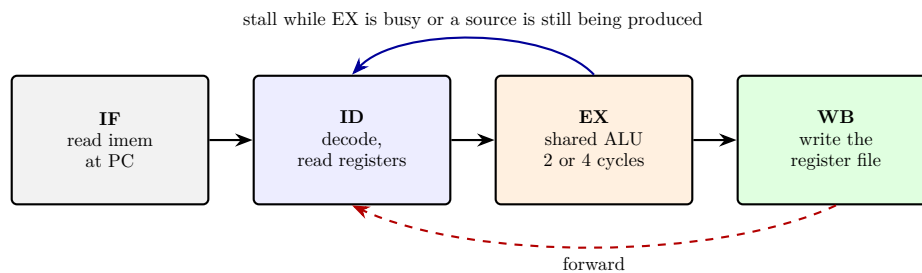
صورت آزمایش می‌گوید ماشین یک حافظه‌ی ۳۲ کلمه‌ای دارد و واحد پایپ‌لاین دستورات را از همان حافظه می‌خواند. هر کلمه ۳۲ بیت است:

opcode [31:28]	rd [27:25]	rs [24:22]	rt [21:19]	— [18:16]	immediate = { imag, real } [15:0]
-------------------	---------------	---------------	---------------	--------------	--------------------------------------

کد	دستور	اثر	توضیح
0	NOP	—	حباب پایپ‌لاین
1	LDI	$rd \leftarrow imm$	تنها راه وارد کردن ثابت
2	MOV	$rd \leftarrow rs$	بدون استفاده از ALU
3	ADD	$rd \leftarrow rs + rt$	دو کلاک جمع‌کننده
4	SUB	$rd \leftarrow rs - rt$	دو کلاک جمع‌کننده
5	MUL	$rd \leftarrow rs \times rt$	چهار کلاک ضرب‌کننده
F	HALT	—	واکشی را متوقف می‌کند

ثبات‌ها هشت‌تا و هرکدام یک عدد مختلط کامل‌اند. LDI و MOV عمداً از ALU استفاده نمی‌کنند؛ این باعث می‌شود در بخش تست بشود اثر خالص پایپ‌لاین را از اثر واحد مشترک جدا اندازه گرفت.

پایپ‌لاین



پیامد جالب قید تک‌واحدی این است که مرحله‌ی EX خودش نمی‌تواند پایپ‌لاین شود: وقتی فقط یک ضرب‌کننده وجود دارد، تنها یک حاصل‌ضرب می‌تواند در جریان باشد. اما IF و ID و WB همچنان با EX هم‌پوشانی دارند — در حالی که یک ضرب چهار کلاک طول می‌کشد، دستور بعدی واکنشی و کدگشایی شده و نتیجه‌ی دستور قبلی در حال نوشته شدن است. به بیان دیگر EX گلوگاه گذرده‌ی است، ولی این یک انتخاب اعلام‌شده است نه یک اشتباه. دو قفل داخلی درستی را تضمین می‌کنند:

الف) قفل منبع. مرحله‌ی EX فقط وقتی دستور تازه می‌پذیرد که آزاد باشد (یا در همین کلاک آزاد شود). بنابراین یک عملیات طولانی، کدگشایی و واکنشی را پشت خودش متوقف می‌کند.

ب) قفل داده و ارسال به جلو. دستوری که ثابت می‌دانش هنوز در EX در حال تولید است، در ID منتظر می‌ماند. به محض اینکه تولیدکننده به WB برسد، مقدار مستقیماً به مرحله‌ی کدگشایی ارسال به جلو می‌شود. نتیجه این است که انتظار دقیقاً به اندازه‌ای است که باید باشد و نه یک کلاک بیشتر — عددی که در بخش تست اندازه‌گیری شده است.

ج) اعلام پایان کار. سیگنال halted فقط یک چیز می‌گوید: واکنشی متوقف شده است. ولی در همان لحظه ممکن است آخرین دستور هنوز در EX باشد، یا نتیجه‌اش در ثبات WB منتظر لبه‌ی بعدی کلاک نشسته باشد. هر کسی که با دیدن halted فایل ثبات را بخواند، مقدار قدیمی می‌بیند.

به همین دلیل خروجی جداگانه‌ای به نام finished وجود دارد که علاوه بر توقف واکنشی، خالی بودن پایپ‌لاین را هم شرط می‌کند:

```
assign finished = halted && !ex_v && !wb_v;
```

این تمایز در نگاه اول بی‌اهمیت به نظر می‌رسد ولی دقیقاً همان چیزی است که در اولین نسخه‌ی تست‌بنج باعث خطا شد: تست با دیدن halted ثبات‌ها را می‌خواند و نتیجه‌ی آخرین دستور را از دست می‌داد. راه درست، اصلاح واسط بود نه اضافه کردن تاخیر به تست.

نکته‌ی ظریف دیگری که ارزش گفتن دارد: ارسال به جلو از WB به ID لازم است، نه یک بهینه‌سازی اختیاری. آرایه‌ی ثبات‌ها در لبه‌ی کلاک نوشته می‌شود و خواندن آن ترکیبی است، پس دستوری که در همان کلاک نوشتن، همان ثبات را می‌خواند مقدار قدیمی را می‌بیند. بدون این مسیر، برنامه‌های وابسته پاسخ غلط می‌دادند.

```

1  `include "complex_pkg.vh"
2
3  module complex_cpu (
4      input wire      Clk,
5      input wire      RstN,
6      input wire      run,          // 0 = hold (used while loading)
7
8      // program load port
9      input wire      prog_we,
10     input wire [MEM_AW-1:0] prog_addr,
11     input wire [IW-1:0] prog_data,
12
13     // observation
14     input wire [REG_AW-1:0] dbg_raddr,
15     output wire [CX_WORD-1:0] dbg_rdata,
16     output reg [MEM_AW-1:0] pc,
17     output reg      halted,        // fetching has stopped
18     output wire      finished,     // ... and the pipeline is empty
  
```

```

19 output reg          ovf_sticky,
20 output wire         alu_busy,
21 output wire         ex_busy,
22 output wire         wb_we,
23 output wire [ `REG_AW-1:0] wb_rd,
24 output wire [ `CX_WORD-1:0] wb_data
25 );
26 // ----- IF -----
27 wire [ `IW-1:0] if_instr;
28
29 imem u_imem (
30     .Clk(Clk), .we(prog_we), .waddr(prog_addr), .wdata(prog_data),
31     .raddr(pc), .rdata(if_instr)
32 );
33
34 // ----- IF / ID -----
35 reg          id_v;
36 reg [ `IW-1:0] id_ir;
37
38 wire [3:0]      id_op = id_ir[31:28];
39 wire [ `REG_AW-1:0] id_rd = id_ir[27:25];
40 wire [ `REG_AW-1:0] id_rs = id_ir[24:22];
41 wire [ `REG_AW-1:0] id_rt = id_ir[21:19];
42 wire [ `CX_WORD-1:0] id_imm = id_ir[16:0];
43
44 wire id_is_alu = (id_op == `OP_ADD) || (id_op == `OP_SUB) || (id_op == `OP_MUL);
45 wire id_is_halt = (id_op == `OP_HALT);
46 wire id_writes = id_is_alu || (id_op == `OP_LDI) || (id_op == `OP_MOV);
47 wire id_uses_rs = id_is_alu || (id_op == `OP_MOV);
48 wire id_uses_rt = id_is_alu;
49
50 wire [1:0] id_alu_op = (id_op == `OP_SUB) ? `ALU_SUB :
51                     (id_op == `OP_MUL) ? `ALU_MUL : `ALU_ADD;
52
53 // ----- ID -----
54 wire [ `CX_WORD-1:0] rf_rd1, rf_rd2;
55 reg          wb_v;
56 reg [ `REG_AW-1:0] wb_a;
57 reg [ `CX_WORD-1:0] wb_d;
58
59 assign wb_we = wb_v;
60 assign wb_rd = wb_a;
61 assign wb_data = wb_d;
62
63 regfile u_rf (
64     .Clk(Clk), .RstN(RstN),
65     .we(wb_v), .wa(wb_a), .wd(wb_d),
66     .ra1(id_rs), .ra2(id_rt), .rd1(rf_rd1), .rd2(rf_rd2),
67     .ra_dbg(dbg_raddr), .rd_dbg(dbg_rdata)
68 );
69
70 // Write-back forwarding: the register array is written on this edge, so a
71 // combinational read of the same address still returns the old value.
72 wire [ `CX_WORD-1:0] rs_val = (wb_v && wb_a == id_rs) ? wb_d : rf_rd1;
73 wire [ `CX_WORD-1:0] rt_val = (wb_v && wb_a == id_rt) ? wb_d : rf_rd2;
74
75 // ----- ID / EX -----
76 reg          ex_v;
77 reg          ex_is_alu;
78 reg          ex_writes;
79 reg [ `REG_AW-1:0] ex_rd;
80 reg [ `CX_WORD-1:0] ex_simple; // result of LDI / MOV, which need no ALU
81
82 reg          alu_start;
83 reg [1:0]    alu_op;
84 reg [ `CX_WORD-1:0] alu_a, alu_b;
85 wire [ `CX_WORD-1:0] alu_y;
86 wire          alu_done, alu_ovf;
87
88 complex_alu u_alu (
89     .Clk(Clk), .RstN(RstN),
90     .start(alu_start), .op(alu_op), .a(alu_a), .b(alu_b),
91     .y(alu_y), .ovf(alu_ovf), .busy(alu_busy), .done(alu_done)
92 );
93
94 assign ex_busy = ex_v;
95
96 assign finished = halted && !ex_v && !wb_v;
97
98 // ----- control -----
99 wire ex_completing = ex_v && (!ex_is_alu || alu_done);
100 wire ex_can_accept = !ex_v || ex_completing;
101
102 wire hazard = ex_v && ex_writes &&
103             ((id_uses_rs && ex_rd == id_rs) ||
104              (id_uses_rt && ex_rd == id_rt));
105

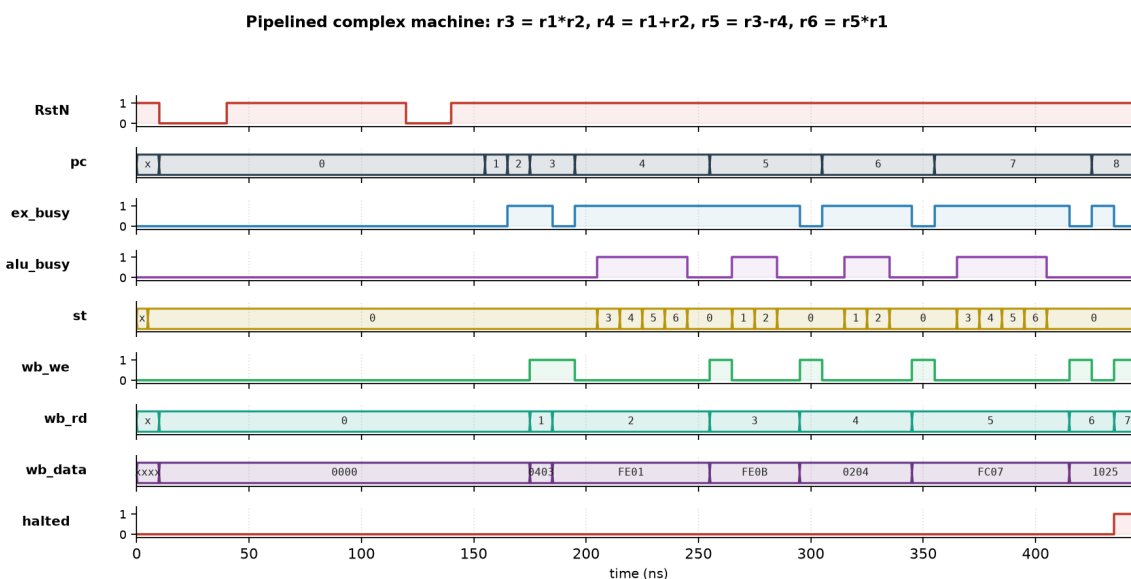
```

```

106 wire halting = id_v && id_is_halt && ex_can_accept;
107 wire id_go    = id_v && ex_can_accept && !hazard;
108 wire if_go    = !halted && !halting && (!id_v || id_go);
109
110 // ----- sequential -----
111 always @(posedge Clk or negedge RstN) begin
112     if (!RstN) begin
113         pc          <= {`MEM_AW{1'b0}};
114         id_v         <= 1'b0;
115         id_ir        <= {`IW{1'b0}};
116         ex_v         <= 1'b0;
117         ex_is_alu    <= 1'b0;
118         ex_writes    <= 1'b0;
119         ex_rd        <= {`REG_AW{1'b0}};
120         ex_simple    <= {`CX_WORD{1'b0}};
121         alu_start    <= 1'b0;
122         alu_op       <= 2'd0;
123         alu_a        <= {`CX_WORD{1'b0}};
124         alu_b        <= {`CX_WORD{1'b0}};
125         wb_v         <= 1'b0;
126         wb_a         <= {`REG_AW{1'b0}};
127         wb_d         <= {`CX_WORD{1'b0}};
128         halted       <= 1'b0;
129         ovf_sticky   <= 1'b0;
130     end else if (run) begin
131         alu_start <= 1'b0;
132         wb_v      <= 1'b0;
133
134         // ---- EX -> WB ----
135         if (ex_completing) begin
136             wb_v <= ex_writes;
137             wb_a <= ex_rd;
138             wb_d <= ex_is_alu ? alu_y : ex_simple;
139             if (ex_is_alu && alu_ovf) ovf_sticky <= 1'b1;
140             ex_v <= 1'b0;
141         end
142
143         if (id_go && !id_is_halt) begin
144             ex_v <= 1'b1;
145             ex_is_alu <= id_is_alu;
146             ex_writes <= id_writes;
147             ex_rd <= id_rd;
148             ex_simple <= (id_op == `OP_LDI) ? id_imm : rs_val;
149             if (id_is_alu) begin
150                 alu_start <= 1'b1;
151                 alu_op <= id_alu_op;
152                 alu_a <= rs_val;
153                 alu_b <= rt_val;
154             end
155         end
156
157         // ---- IF -> ID ----
158         if (halting) begin
159             halted <= 1'b1;
160             id_v <= 1'b0;
161         end else if (if_go) begin
162             id_v <= 1'b1;
163             id_ir <= if_instr;
164             pc <= pc + 1'b1;
165         end else if (id_go) begin
166             id_v <= 1'b0;
167         end
168     end
169 end
170 endmodule

```

شکل موج



برنامه‌ی نمایش داده‌شده این است:

دستور	محاسبه	مقدار نوشته‌شده
LDI r1, 3+4j	—	0x0403
LDI r2, 1-2j	—	0xFE01
MUL r3, r1, r2	$(3 + 4j)(1 - 2j) = 11 - 2j$	0xFE0B
ADD r4, r1, r2	$(3 + 4j) + (1 - 2j) = 4 + 2j$	0x0204
SUB r5, r3, r4	$(11 - 2j) - (4 + 2j) = 7 - 4j$	0xFC07
MUL r6, r5, r1	$(7 - 4j)(3 + 4j) = 37 + 16j$	0x1025
MOV r7, r6	—	0x1025

سه چیز در شکل موج قابل دیدن است. اول، سیگنال st که حالت داخلی ALU است: برای هر MUL دنباله‌ی ۳، ۴، ۵، ۶ (یعنی M1 تا M4) و برای هر ADD و SUB دنباله‌ی ۱، ۲ دیده می‌شود — همان زمان‌بندی جدول بالا. دوم، alu_busy هرگز دو عملیات را هم‌زمان نشان نمی‌دهد، که همان قید تک‌واحدی است. سوم، pc در حالی جلو می‌رود که ALU هنوز مشغول است: این خود هم‌پوشانی پایپ‌لاین است.

توجه کنید که دو نوشتن آخر (r6 و r7) پشت سر هم و بدون فاصله انجام می‌شوند، چون MOV از ALU استفاده نمی‌کند و بلافاصله پس از آزاد شدن EX وارد آن می‌شود.

تست‌ها

پنج تست وجود دارد که اولین آن‌ها اصلاً شبیه‌سازی نیست.

(۱) **بررسی منبع.** قید «بیش از یک بار استفاده نشود» یک ویژگی متن مدار است، نه یک ویژگی شکل موج. هیچ شبیه‌سازی‌ای نمی‌تواند نشان دهد که دو ضرب‌کننده در مدار وجود ندارد — مداری با دو ضرب‌کننده همان جواب‌های درست را می‌دهد. بنابراین اسکرپت تست پیش از هر شبیه‌سازی، تعداد نمونه‌سازی‌های mul_unit، add_unit و complex_alu را در کل RTL می‌شمارد و اگر هرکدام بیش از یکی باشد تست را رد می‌کند.

(۲ و ۳) **بخش الف و بخش ب.** هر کدام یک مدل مرجع دارند که با حساب صحیح ساده نوشته شده — بدون ماشین حالت و بدون زمان‌بندی — و بنابراین هیچ اشتباهی را با طراحی شریک نمی‌شود. پوشش شامل تمام ترکیب‌های

نه مقدار مرزی روی هر چهار مؤلفه (یعنی $6561 = 9^4$ حالت برای هر عملیات) به‌علاوه‌ی یک جاروب تصادفی است. علاوه بر مقدار عددی، پرچم اشباع و تعداد کلاک هر عملیات هم بررسی می‌شود.

```

1 `timescale 1ns/1ps
2 `include "complex_pkg.vh"
3
4 module tb_complex_addsub;
5     reg                Clk = 1'b0, RstN = 1'b1;
6     reg                start = 1'b0;
7     reg [1:0]          op = 2'd0;
8     reg [`CX_WORD-1:0] a = 16'd0, b = 16'd0;
9     wire [`CX_WORD-1:0] y;
10    wire                ovf, busy, done;
11
12    complex_alu dut (
13        .Clk(Clk), .RstN(RstN), .start(start), .op(op), .a(a), .b(b),
14        .y(y), .ovf(ovf), .busy(busy), .done(done)
15    );
16
17    always #5 Clk = ~Clk;
18
19    integer pass = 0, fail = 0;
20    integer cyc = 0;
21    always @(posedge Clk) cyc = cyc + 1;
22
23    integer i, j, k, m, n, t0, seed;
24    integer ar, ai, br, bi, er, ei;
25    reg      eovf;
26
27    integer corner [0:8];
28
29    function integer sat8i;
30        input integer v;
31        begin
32            if (v > 127) sat8i = 127;
33            else if (v < -128) sat8i = -128;
34            else sat8i = v;
35        end
36    endfunction
37
38    function outside;
39        input integer v;
40        begin
41            outside = (v > 127) || (v < -128);
42        end
43    endfunction
44
45    task automatic chk;
46        input [511:0] name;
47        input         ok;
48        begin
49            if (ok) pass = pass + 1;
50            else begin
51                fail = fail + 1;
52                $display(" FAIL @%0t: %0s op=%0d a=%h b=%h y=%h (want re=%0d im=%0d)",
53                    $time, name, op, a, b, y, er, ei);
54            end
55        end
56    endtask
57
58    // Drive one operation and return with y valid.
59    task automatic run_op;
60        input [1:0] o;
61        input [`CX_WORD-1:0] A;
62        input [`CX_WORD-1:0] B;
63        begin
64            @(negedge Clk); op = o; a = A; b = B; start = 1'b1;
65            @(negedge Clk); start = 1'b0; t0 = cyc;
66            while (!done) @(negedge Clk);
67        end
68    endtask
69
70    // Model + compare for one operand pair.
71    task automatic check_pair;
72        input [1:0] o;
73        input [`CX_WORD-1:0] A;
74        input [`CX_WORD-1:0] B;
75        begin
76            ar = $signed(A[7:0]); ai = $signed(A[15:8]);
77            br = $signed(B[7:0]); bi = $signed(B[15:8]);
78
79            if (o == `ALU_SUB) begin
80                er = sat8i(ar - br); ei = sat8i(ai - bi);
81                eovf = outside(ar - br) || outside(ai - bi);
82            else begin
83                er = sat8i(ar + br); ei = sat8i(ai + bi);

```

```

84         eovf = outside(ar + br) || outside(ai + bi);
85     end
86
87     run_op(o, A, B);
88     chk("real part", y[7:0] === er[7:0]);
89     chk("imaginary part", y[15:8] === ei[7:0]);
90     chk("overflow flag", ovf === eovf);
91 end
92 endtask
93
94 initial begin
95     corner[0] = -128; corner[1] = -127; corner[2] = -64;
96     corner[3] = -1; corner[4] = 0; corner[5] = 1;
97     corner[6] = 5; corner[7] = 64; corner[8] = 127;
98
99     @(negedge Clk); RstN = 1'b0;
100    repeat (3) @(negedge Clk); RstN = 1'b1;
101    @(negedge Clk);
102
103    chk("idle after reset", busy === 1'b0 && done === 1'b0);
104
105    // ---- a worked example, so the report has something concrete ----
106    // (3 + 4j) + (-1 + 2j) = (2 + 6j)
107    run_op(`ALU_ADD, {8'sd4, 8'sd3}, {8'sd2, -8'sd1});
108    chk("(3+4j)+(-1+2j) = 2+6j", y === {8'sd6, 8'sd2});
109    // (3 + 4j) - (-1 + 2j) = (4 + 2j)
110    run_op(`ALU_SUB, {8'sd4, 8'sd3}, {8'sd2, -8'sd1});
111    chk("(3+4j)-(-1+2j) = 4+2j", y === {8'sd2, 8'sd4});
112
113    // ---- latency: exactly two passes through the shared adder ----
114    run_op(`ALU_ADD, {8'sd1, 8'sd1}, {8'sd1, 8'sd1});
115    chk("add occupies the adder for 2 cycles", (cyc - t0) == 2);
116    run_op(`ALU_SUB, {8'sd1, 8'sd1}, {8'sd1, 8'sd1});
117    chk("sub occupies the adder for 2 cycles", (cyc - t0) == 2);
118
119    // ---- every corner combination, both operations ----
120    for (i = 0; i < 9; i = i + 1)
121        for (j = 0; j < 9; j = j + 1)
122            for (k = 0; k < 9; k = k + 1)
123                for (m = 0; m < 9; m = m + 1) begin
124                    check_pair(`ALU_ADD, {corner[j][7:0], corner[i][7:0]},
125                                {corner[m][7:0], corner[k][7:0]});
126                    check_pair(`ALU_SUB, {corner[j][7:0], corner[i][7:0]},
127                                {corner[m][7:0], corner[k][7:0]});
128                end
129
130    // ---- random sweep ----
131    seed = 32'd20260826;
132    for (n = 0; n < 1000; n = n + 1) begin
133        check_pair(n[0] ? `ALU_SUB : `ALU_ADD,
134                    $random(seed), $random(seed));
135    end
136
137    $display("tb_complex_addsub: Passed: %0d, Failed: %0d", pass, fail);
138    if (fail != 0) $fatal(1);
139    $finish;
140 end
141 endmodule

```

```

1 `timescale 1ns/1ps
2 `include "complex_pkg.vh"
3
4 module tb_complex_mul;
5     reg Clk = 1'b0, RstN = 1'b1;
6     reg start = 1'b0;
7     reg [1:0] op = `ALU_MUL;
8     reg [`CX_WORD-1:0] a = 16'd0, b = 16'd0;
9     wire [`CX_WORD-1:0] y;
10    wire ovf, busy, done;
11
12    complex_alu dut (
13        .Clk(Clk), .RstN(RstN), .start(start), .op(op), .a(a), .b(b),
14        .y(y), .ovf(ovf), .busy(busy), .done(done)
15    );
16
17    always #5 Clk = ~Clk;
18
19    integer pass = 0, fail = 0;
20    integer cyc = 0;
21    always @(posedge Clk) cyc = cyc + 1;
22
23    always @(posedge Clk)
24        if (RstN && start && busy) begin
25            $display(" FAIL %0t: start asserted while the ALU is busy", $time);
26            fail = fail + 1;
27        end

```

```

28
29 integer i, j, k, m, n, t0, seed;
30 integer ar, ai, br, bi, er, ei, raw_re, raw_im;
31 reg eovf;
32
33 integer corner [0:8];
34
35 function integer sat8i;
36   input integer v;
37   begin
38     if (v > 127) sat8i = 127;
39     else if (v < -128) sat8i = -128;
40     else sat8i = v;
41   end
42 endfunction
43
44 function outside;
45   input integer v;
46   begin
47     outside = (v > 127) || (v < -128);
48   end
49 endfunction
50
51 task automatic chk;
52   input [511:0] name;
53   input ok;
54   begin
55     if (ok) pass = pass + 1;
56     else begin
57       fail = fail + 1;
58       $display(" FAIL @%0t: %0s a=%h b=%h y=%h (want re=%0d im=%0d)",
59               $time, name, a, b, y, er, ei);
60     end
61   end
62 endtask
63
64 task automatic run_mul;
65   input [`CX_WORD-1:0] A;
66   input [`CX_WORD-1:0] B;
67   begin
68     @(negedge Clk); a = A; b = B; op = `ALU_MUL; start = 1'b1;
69     @(negedge Clk); start = 1'b0; t0 = cyc;
70     while (!done) @(negedge Clk);
71   end
72 endtask
73
74 task automatic check_pair;
75   input [`CX_WORD-1:0] A;
76   input [`CX_WORD-1:0] B;
77   begin
78     ar = $signed(A[7:0]); ai = $signed(A[15:8]);
79     br = $signed(B[7:0]); bi = $signed(B[15:8]);
80
81     raw_re = ar*br - ai*bi;
82     raw_im = ar*bi + ai*br;
83     er = sat8i(raw_re);
84     ei = sat8i(raw_im);
85     eovf = outside(raw_re) || outside(raw_im);
86
87     run_mul(A, B);
88     chk("real part", y[7:0] === er[7:0]);
89     chk("imaginary part", y[15:8] === ei[7:0]);
90     chk("overflow flag", ovf === eovf);
91   end
92 endtask
93
94 initial begin
95   corner[0] = -128; corner[1] = -127; corner[2] = -64;
96   corner[3] = -1; corner[4] = 0; corner[5] = 1;
97   corner[6] = 5; corner[7] = 11; corner[8] = 127;
98
99   @(negedge Clk); RstN = 1'b0;
100  repeat (3) @(negedge Clk); RstN = 1'b1;
101  @(negedge Clk);
102
103  // ---- worked examples ----
104  // (3 + 4j)(1 - 2j) = 3 - 6j + 4j + 8 = 11 - 2j
105  run_mul({8'sd4, 8'sd3}, {-8'sd2, 8'sd1});
106  chk("(3+4j)(1-2j) = 11-2j", y === {-8'sd2, 8'sd11});
107  // j * j = -1
108  run_mul({8'sd1, 8'sd0}, {8'sd1, 8'sd0});
109  chk("j*j = -1", y === {8'sd0, -8'sd1});
110  // (5 + 0j)(0 + 3j) = 15j
111  run_mul({8'sd0, 8'sd5}, {8'sd3, 8'sd0});
112  chk("5 * 3j = 15j", y === {8'sd15, 8'sd0});
113  // saturation: (100 + 0j)(0 + 100j) = 10000j -> clamped to 127j
114  run_mul({8'sd0, 8'sd100}, {8'sd100, 8'sd0});

```

```

115     chk("100 * 100j saturates to 127j", y == {8'sd127, 8'sd0} && ovf == 1'b1);
116
117     // ---- latency: one cycle per real product ----
118     run_mul({8'sd1, 8'sd1}, {8'sd1, 8'sd1});
119     chk("multiply occupies the multiplier for 4 cycles", (cyc - t0) == 4);
120
121     // ---- every corner combination ----
122     for (i = 0; i < 9; i = i + 1)
123     for (j = 0; j < 9; j = j + 1)
124     for (k = 0; k < 9; k = k + 1)
125     for (m = 0; m < 9; m = m + 1)
126         check_pair({corner[j][7:0], corner[i][7:0]},
127                   {corner[m][7:0], corner[k][7:0]});
128
129     // ---- random sweep ----
130     seed = 32'd1405;
131     for (n = 0; n < 1000; n = n + 1)
132         check_pair($random(seed), $random(seed));
133
134     $display("tb_complex_mul: Passed: %0d, Failed: %0d", pass, fail);
135     if (fail != 0) $fatal(1);
136     $finish;
137 end
138 endmodule

```

۴) ماشین در برابر مدل مرجع. مدل یک مفسر ساده‌ی همان برنامه است: بدون پایپ‌لاین، بدون توقف و بدون واحد مشترک. توافق با آن یعنی قفل‌ها و مسیر ارسال به جلو دقیقاً همان نتیجه‌ای را می‌سازند که اجرای دستورات یکی‌یکی می‌ساخت.

بیشتر کار را برنامه‌های تصادفی انجام می‌دهند. برنامه‌ی دست‌نویس فقط الگوهای وابستگی‌ای را پوشش می‌دهد که نویسنده‌اش به آن‌ها فکر کرده؛ صد و پنجاه برنامه‌ی تصادفی زنجیره‌های وابستگی، استفاده‌ی مجدد از ثبات‌ها و هم‌نامی عملوندها را پوشش می‌دهند که هیچ‌کس آن‌ها را عمداً نمی‌نویسد.

```

1  `timescale 1ns/1ps
2  `include "complex_pkg.vh"
3
4  module tb_complex_cpu;
5      reg          Clk = 1'b0, RstN = 1'b1, run_en = 1'b0;
6      reg          prog_we = 1'b0;
7      reg [MEM_AW-1:0] prog_addr = 5'd0;
8      reg [IW-1:0]   prog_data = 32'd0;
9      reg [REG_AW-1:0] dbg_raddr = 3'd0;
10
11     wire [CX_WORD-1:0] dbg_rdata;
12     wire [MEM_AW-1:0] pc;
13     wire             halted, finished, ovf_sticky, alu_busy, ex_busy, wb_we;
14     wire [REG_AW-1:0] wb_rd;
15     wire [CX_WORD-1:0] wb_data;
16
17     complex_cpu dut (
18         .Clk(Clk), .RstN(RstN), .run(run_en),
19         .prog_we(prog_we), .prog_addr(prog_addr), .prog_data(prog_data),
20         .dbg_raddr(dbg_raddr), .dbg_rdata(dbg_rdata),
21         .pc(pc), .halted(halted), .finished(finished), .ovf_sticky(ovf_sticky),
22         .alu_busy(alu_busy), .ex_busy(ex_busy),
23         .wb_we(wb_we), .wb_rd(wb_rd), .wb_data(wb_data)
24     );
25
26     always #5 Clk = ~Clk;
27
28     integer pass = 0, fail = 0;
29     integer i, n, prog_len, seed, guard;
30
31     reg [IW-1:0]   prog [0:MEM_WORDS-1];
32     reg [CX_WORD-1:0] mreg [0:7]; // model register file
33     reg          movf; // model sticky overflow
34
35     task automatic chk;
36         input [511:0] name;
37         input         ok;
38         begin
39             if (ok) pass = pass + 1;
40             else begin
41                 fail = fail + 1;
42                 $display(" FAIL @%0t: %0s", $time, name);
43             end
44         end
45     endtask
46
47     function [IW-1:0] enc;
48         input [3:0]   o;

```

```

49     input [`REG_AW-1:0] rd;
50     input [`REG_AW-1:0] rs;
51     input [`REG_AW-1:0] rt;
52     input [`CX_WORD-1:0] imm;
53     begin
54         enc = {o, rd, rs, rt, 3'b000, imm};
55     end
56 endfunction
57
58 function integer sat8i;
59     input integer v;
60     begin
61         if (v > 127) sat8i = 127;
62         else if (v < -128) sat8i = -128;
63         else sat8i = v;
64     end
65 endfunction
66
67 function outside;
68     input integer v;
69     begin
70         outside = (v > 127) || (v < -128);
71     end
72 endfunction
73
74 // ---- the reference model: run the program one instruction at a time ----
75 task automatic model_run;
76     integer p, ar, ai, br, bi, rr, ri, rr8, ri8;
77     reg [`IW-1:0] ins;
78     reg [3:0] o;
79     reg [`REG_AW-1:0] rd, rs, rt;
80     integer g;
81     begin
82         for (g = 0; g < 8; g = g + 1) mreg[g] = {'CX_WORD{1'b0}};
83         movf = 1'b0;
84         p = 0;
85         g = 0;
86         while (p < `MEM_WORDS && g < 4*`MEM_WORDS) begin
87             g = g + 1;
88             ins = prog[p];
89             o = ins[31:28];
90             rd = ins[27:25];
91             rs = ins[24:22];
92             rt = ins[21:19];
93
94             ar = $signed(mreg[rs][7:0]); ai = $signed(mreg[rs][15:8]);
95             br = $signed(mreg[rt][7:0]); bi = $signed(mreg[rt][15:8]);
96
97             case (o)
98                 `OP_HALT: p = `MEM_WORDS; // stop
99                 `OP_NOP: p = p + 1;
100                 `OP_LDI: begin mreg[rd] = ins[15:0]; p = p + 1; end
101                 `OP_MOV: begin mreg[rd] = mreg[rs]; p = p + 1; end
102                 `OP_ADD, `OP_SUB, `OP_MUL: begin
103                     if (o == `OP_ADD) begin rr = ar + br; ri = ai + bi; end
104                     else if (o == `OP_SUB) begin rr = ar - br; ri = ai - bi; end
105                     else begin rr = ar*br - ai*bi; ri = ar*bi + ai*br; end
106                     if (outside(rr) || outside(ri)) movf = 1'b1;
107                     rr8 = sat8i(rr);
108                     ri8 = sat8i(ri);
109                     mreg[rd] = {ri8[7:0], rr8[7:0]};
110                     p = p + 1;
111                 end
112                 default: p = p + 1; // unused opcode acts as NOP
113             endcase
114         end
115     end
116 endtask
117
118 // ---- load a program, reset the pipeline, run it to completion ----
119 task automatic hw_run;
120     output integer cycles;
121     integer c;
122     begin
123         run_en = 1'b0;
124         @(negedge Clk);
125         for (c = 0; c < `MEM_WORDS; c = c + 1) begin
126             prog_we = 1'b1; prog_addr = c[`MEM_AW-1:0]; prog_data = prog[c];
127             @(negedge Clk);
128         end
129         prog_we = 1'b0;
130
131         RstN = 1'b0; @(negedge Clk); @(negedge Clk); RstN = 1'b1;
132         @(negedge Clk);
133
134         run_en = 1'b1;
135         c = 0;

```

```

136 // `finished` (not `halted`) is the safe point to read results:
137 // it also requires the last write-back to have landed.
138 while (!finished && c < 4000) begin
139     @(negedge Clk);
140     c = c + 1;
141 end
142 cycles = c;
143 run_en = 1'b0;
144 end
145 endtask
146
147 task automatic compare_state;
148     input [511:0] tag;
149     integer g;
150     reg [`CX_WORD-1:0] got;
151     begin
152         for (g = 0; g < 8; g = g + 1) begin
153             dbg_raddr = g[`REG_AW-1:0]; #1; got = dbg_rdata;
154             if (got !== mreg[g]) begin
155                 fail = fail + 1;
156                 $display(" FAIL @%0t: %0s r%0d = %h, model says %h",
157                     $time, tag, g, got, mreg[g]);
158             end else pass = pass + 1;
159         end
160         chk("sticky overflow flag", ovf_sticky === movf);
161     end
162 endtask
163
164 task automatic rd_hw;
165     input [`REG_AW-1:0] r;
166     output [`CX_WORD-1:0] v;
167     begin
168         dbg_raddr = r; #1; v = dbg_rdata;
169     end
170 endtask
171
172 task automatic fill_halt;
173     integer g;
174     begin
175         for (g = 0; g < `MEM_WORDS; g = g + 1)
176             prog[g] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
177     end
178 endtask
179
180 integer cycles;
181 reg [`CX_WORD-1:0] tmp;
182
183 initial begin
184     @(negedge Clk); RstN = 1'b0;
185     repeat (3) @(negedge Clk); RstN = 1'b1;
186
187     // ----- a demo program --
188     fill_halt;
189     prog[0] = enc(`OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd4, 8'sd3});
190     prog[1] = enc(`OP_LDI, 3'd2, 3'd0, 3'd0, {8'sd2, 8'sd1});
191     prog[2] = enc(`OP_MUL, 3'd3, 3'd1, 3'd2, 16'd0);
192     prog[3] = enc(`OP_ADD, 3'd4, 3'd1, 3'd2, 16'd0);
193     prog[4] = enc(`OP_SUB, 3'd5, 3'd3, 3'd4, 16'd0);
194     prog[5] = enc(`OP_MUL, 3'd6, 3'd5, 3'd1, 16'd0);
195     prog[6] = enc(`OP_MOV, 3'd7, 3'd6, 3'd0, 16'd0);
196     prog[7] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
197
198     model_run;
199     hw_run(cycles);
200     chk("demo program halted", halted === 1'b1);
201     compare_state("demo program");
202     rd_hw(3'd3, tmp); chk("demo: r3 = 11 - 2j", tmp === {-8'sd2, 8'sd11});
203     rd_hw(3'd5, tmp); chk("demo: r5 = 7 - 4j", tmp === {-8'sd4, 8'sd7});
204     rd_hw(3'd6, tmp); chk("demo: r6 = 37 + 16j", tmp === {8'sd16, 8'sd37});
205     $display(" demo program: %0d instructions in %0d cycles", 8, cycles);
206
207     // ----- a dependency chain: every result feeds the next --
208     fill_halt;
209     prog[0] = enc(`OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd1, 8'sd2});
210     prog[1] = enc(`OP_LDI, 3'd2, 3'd0, 3'd0, {8'sd1, 8'sd1});
211     prog[2] = enc(`OP_ADD, 3'd3, 3'd1, 3'd2, 16'd0);
212     prog[3] = enc(`OP_ADD, 3'd3, 3'd3, 3'd2, 16'd0);
213     prog[4] = enc(`OP_ADD, 3'd3, 3'd3, 3'd2, 16'd0);
214     prog[5] = enc(`OP_MUL, 3'd3, 3'd3, 3'd2, 16'd0);
215     prog[6] = enc(`OP_SUB, 3'd3, 3'd3, 3'd1, 16'd0);
216     prog[7] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
217     model_run;
218     hw_run(cycles);
219     chk("chain program halted", halted === 1'b1);
220     compare_state("back-to-back dependency chain");
221
222     // ----- saturation propagates through a program --

```

```

223 fill_halt;
224 prog[0] = enc(`OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd100, 8'sd100});
225 prog[1] = enc(`OP_LDI, 3'd2, 3'd0, 3'd0, {8'sd100, 8'sd100});
226 prog[2] = enc(`OP_ADD, 3'd3, 3'd1, 3'd2, 16'd0);
227 prog[3] = enc(`OP_MUL, 3'd4, 3'd1, 3'd2, 16'd0);
228 prog[4] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
229 model_run;
230 hw_run(cycles);
231 compare_state("saturating program");
232 chk("overflow was reported", ovf_sticky === 1'b1);
233
234 // ----- random programs -----
235 seed = 32'd8080;
236 for (n = 0; n < 150; n = n + 1) begin
237     fill_halt;
238     prog_len = 4 + ({$random(seed)} % 20);
239     for (i = 0; i < prog_len; i = i + 1) begin
240         case ({$random(seed)} % 8)
241             0, 1: prog[i] = enc(`OP_LDI, ({$random(seed)} % 8), 3'd0, 3'd0,
242                               {$random(seed)});
243             2: prog[i] = enc(`OP_MOV, ({$random(seed)} % 8),
244                               ({$random(seed)} % 8), 3'd0, 16'd0);
245             3: prog[i] = enc(`OP_NOP, 3'd0, 3'd0, 3'd0, 16'd0);
246             4, 5: prog[i] = enc(`OP_ADD, ({$random(seed)} % 8),
247                               ({$random(seed)} % 8), ({$random(seed)} % 8), 16'd0);
248             6: prog[i] = enc(`OP_SUB, ({$random(seed)} % 8),
249                               ({$random(seed)} % 8), ({$random(seed)} % 8), 16'd0);
250             7: prog[i] = enc(`OP_MUL, ({$random(seed)} % 8),
251                               ({$random(seed)} % 8), ({$random(seed)} % 8), 16'd0);
252         endcase
253     end
254     prog[prog_len] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
255
256     model_run;
257     hw_run(cycles);
258     chk("random program halted", halted === 1'b1);
259     compare_state("random program");
260 end
261
262 $display("tb_complex_cpu: Passed: %0d, Failed: %0d", pass, fail);
263 if (fail != 0) $fatal(1);
264 $finish;
265
266 end
endmodule

```

۵) ادعاهای ساختاری پایپ‌لاین. مقایسه با مدل مرجع نشان می‌دهد ماشین عدد درست را می‌دهد، ولی دربارۀ این‌که چرا این‌قدر طول می‌کشد چیزی نمی‌گوید — ماشینی که بی‌سروصدا هر دستور را جداگانه و بدون هم‌پوشانی اجرا کند هم همان تست را پاس می‌کند. تست پنجم دقیقاً همین را اندازه می‌گیرد:

سناریو	کلاک	نتیجه‌گیری
۱۶ دستور تک‌کلاکی مستقل	۱۹	حدود یک کلاک برای هر دستور؛ بدون هم‌پوشانی دست‌کم 3×16 لازم بود
۱۶ ضرب مستقل	۱۰۲	گلوگاه، ضرب‌کننده‌ی مشترک است نه پایپ‌لاین
۱۶ ضرب زنجیره‌ای وابسته	۱۱۷	هزینه‌ی قفل داده

سطر آخر عدد قشنگی می‌دهد: اختلاف $15 = 117 - 102$ کلاک است و تعداد جفت‌های وابسته هم دقیقاً ۱۵ تاست. یعنی قفل داده دقیقاً یک کلاک برای هر وابستگی هزینه دارد — همان یک کلاکی که مصرف‌کننده منتظر می‌ماند تا تولیدکننده از EX به WB برود و مسیر ارسال به جلو در دسترس شود. این تساوی در تست‌بنچ به‌صورت یک شرط دقیق نوشته شده، نه به‌صورت یک بازه.

```

1 `timescale 1ns/1ps
2 `include "complex_pkg.vh"
3
4 module tb_complex_cpu_spec;
5     reg Clk = 1'b0, RstN = 1'b1, run_en = 1'b0;
6     reg prog_we = 1'b0;
7     reg [MEM_AW-1:0] prog_addr = 5'd0;
8     reg [IW-1:0] prog_data = 32'd0;
9     reg [REG_AW-1:0] dbg_raddr = 3'd0;
10

```

```

11 wire [`CX_WORD-1:0] dbg_rdata;
12 wire [`MEM_AW-1:0] pc;
13 wire halted, finished, ovf_sticky, alu_busy, ex_busy, wb_we;
14 wire [`REG_AW-1:0] wb_rd;
15 wire [`CX_WORD-1:0] wb_data;
16
17 complex_cpu dut (
18     .Clk(Clk), .RstN(RstN), .run(run_en),
19     .prog_we(prog_we), .prog_addr(prog_addr), .prog_data(prog_data),
20     .dbg_raddr(dbg_raddr), .dbg_rdata(dbg_rdata),
21     .pc(pc), .halted(halted), .finished(finished), .ovf_sticky(ovf_sticky),
22     .alu_busy(alu_busy), .ex_busy(ex_busy),
23     .wb_we(wb_we), .wb_rd(wb_rd), .wb_data(wb_data)
24 );
25
26 always #5 Clk = ~Clk;
27
28 integer pass = 0, fail = 0;
29 integer i, writes, cyc_ldi, cyc_mul_indep, cyc_mul_dep, cyc_nop;
30 reg [`IW-1:0] prog [0:`MEM_WORDS-1];
31 reg [`CX_WORD-1:0] tmp;
32
33 integer wcount = 0;
34 always @(posedge Clk) if (run_en && wb_we) wcount = wcount + 1;
35
36 always @(posedge Clk)
37     if (RstN && run_en && alu_busy && !ex_busy) begin
38         $display(" FAIL @%0t: the ALU is running with no instruction in EX", $time);
39         fail = fail + 1;
40     end
41
42 task automatic chk;
43     input [511:0] name;
44     input ok;
45     begin
46         if (ok) pass = pass + 1;
47         else begin
48             fail = fail + 1;
49             $display(" FAIL @%0t: %0s", $time, name);
50         end
51     end
52 endtask
53
54 function [`IW-1:0] enc;
55     input [3:0] o;
56     input [`REG_AW-1:0] rd;
57     input [`REG_AW-1:0] rs;
58     input [`REG_AW-1:0] rt;
59     input [`CX_WORD-1:0] imm;
60     begin
61         enc = {o, rd, rs, rt, 3'b000, imm};
62     end
63 endfunction
64
65 task automatic fill_halt;
66     integer g;
67     begin
68         for (g = 0; g < `MEM_WORDS; g = g + 1)
69             prog[g] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
70     end
71 endtask
72
73 task automatic hw_run;
74     output integer cycles;
75     integer c;
76     begin
77         run_en = 1'b0;
78         @(negedge Clk);
79         for (c = 0; c < `MEM_WORDS; c = c + 1) begin
80             prog_we = 1'b1; prog_addr = c[`MEM_AW-1:0]; prog_data = prog[c];
81             @(negedge Clk);
82         end
83         prog_we = 1'b0;
84
85         RstN = 1'b0; @(negedge Clk); @(negedge Clk); RstN = 1'b1;
86         @(negedge Clk);
87
88         wcount = 0;
89         run_en = 1'b1;
90         c = 0;
91         while (!finished && c < 4000) begin
92             @(negedge Clk);
93             c = c + 1;
94         end
95         cycles = c;
96         run_en = 1'b0;
97     end

```



```

98   endtask
99
100  task automatic rd_hw;
101      input  [`REG_AW-1:0] r;
102      output [`CX_WORD-1:0] v;
103      begin
104          dbg_raddr = r; #1; v = dbg_rdata;
105      end
106  endtask
107
108  initial begin
109      @(negedge Clk); RstN = 1'b0;
110      repeat (3) @(negedge Clk); RstN = 1'b1;
111
112      fill_halt;
113      for (i = 0; i < 16; i = i + 1)
114          prog[i] = enc(`OP_LDI, i[2:0], 3'd0, 3'd0, {8'd0, i[7:0]});
115      prog[16] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
116      hw_run(cyc_ldi);
117      $display(" 16 independent single-cycle instructions: %0d cycles", cyc_ldi);
118      chk("independent instructions overlap", cyc_ldi < 2*16);
119      chk("all 16 results were written", wcount == 16);
120      rd_hw(3'd7, tmp);
121      chk("last LDI landed", tmp === {8'd0, 8'd15});
122
123      fill_halt;
124      prog[0] = enc(`OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd1, 8'sd2});
125      prog[1] = enc(`OP_LDI, 3'd2, 3'd0, 3'd0, {8'sd1, 8'sd1});
126      for (i = 0; i < 16; i = i + 1)
127          prog[2+i] = enc(`OP_MUL, 3'd3 + (i % 5), 3'd1, 3'd2, 16'd0);
128      prog[18] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
129      hw_run(cyc_mul_indep);
130      $display(" 16 independent multiplies: %0d cycles", cyc_mul_indep);
131      chk("multiplies are limited by the shared multiplier", cyc_mul_indep > 4*16);
132      chk("all 18 results were written", wcount == 18);
133
134      fill_halt;
135      prog[0] = enc(`OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd0, 8'sd1});
136      prog[1] = enc(`OP_LDI, 3'd2, 3'd0, 3'd0, {8'sd1, 8'sd1});
137      for (i = 0; i < 16; i = i + 1)
138          prog[2+i] = enc(`OP_MUL, 3'd2, 3'd2, 3'd1, 16'd0);
139      prog[18] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
140      hw_run(cyc_mul_dep);
141      $display(" 16 dependent multiplies: %0d cycles", cyc_mul_dep);
142      chk("a dependency chain costs more than independent work",
143          cyc_mul_dep > cyc_mul_indep);
144      $display(" cost of the interlock: %0d cycles over %0d dependent pairs",
145          cyc_mul_dep - cyc_mul_indep, 15);
146      chk("the interlock costs exactly one cycle per dependent pair",
147          (cyc_mul_dep - cyc_mul_indep) == 15);
148      rd_hw(3'd2, tmp);
149      chk("x * 1 repeated 16 times is still x", tmp === {8'sd1, 8'sd1});
150
151      fill_halt;
152      prog[0] = enc(`OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd7, 8'sd7});
153      for (i = 1; i < 12; i = i + 1)
154          prog[i] = enc(`OP_NOP, 3'd0, 3'd0, 3'd0, 16'd0);
155      prog[12] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
156      hw_run(cyc_nop);
157      chk("a NOP writes no register", wcount == 1);
158      rd_hw(3'd1, tmp);
159      chk("the LDI before the NOPs still landed", tmp === {8'sd7, 8'sd7});
160
161      fill_halt;
162      prog[0] = enc(`OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd3, 8'sd3});
163      prog[1] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
164      prog[2] = enc(`OP_LDI, 3'd2, 3'd0, 3'd0, {8'sd9, 8'sd9});
165      prog[3] = enc(`OP_LDI, 3'd3, 3'd0, 3'd0, {8'sd9, 8'sd9});
166      hw_run(i);
167      chk("machine halted", halted === 1'b1);
168      chk("only the instruction before HALT wrote", wcount == 1);
169      rd_hw(3'd1, tmp); chk("r1 was written", tmp === {8'sd3, 8'sd3});
170      rd_hw(3'd2, tmp); chk("r2 after HALT untouched", tmp === 16'd0);
171      rd_hw(3'd3, tmp); chk("r3 after HALT untouched", tmp === 16'd0);
172
173      writes = pc;
174      repeat (10) @(negedge Clk);
175      chk("PC frozen after HALT", pc === writes[`MEM_AW-1:0]);
176
177      fill_halt;
178      prog[0] = enc(`OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd0, 8'sd3}); // 3
179      prog[1] = enc(`OP_ADD, 3'd1, 3'd1, 3'd1, 16'd0); // 6
180      prog[2] = enc(`OP_ADD, 3'd1, 3'd1, 3'd1, 16'd0); // 12
181      prog[3] = enc(`OP_MUL, 3'd1, 3'd1, 3'd1, 16'd0); // 144 -> clamps
182      prog[4] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
183      hw_run(i);
184      rd_hw(3'd1, tmp);

```

```

185     chk("rd used as both sources reads the old value",
186         tmp === {8'sd0, 8'sd127});
187     chk("the clamp was reported", ovf_sticky === 1'b1);
188
189     $display("tb_complex_cpu_spec: Passed: %0d, Failed: %0d", pass, fail);
190     if (fail != 0) $fatal(1);
191     $finish;
192 end
193 endmodule

```

نتایج تست

```

=== resource check (one adder, one multiplier) ===
mul_unit instantiated 1 time
add_unit instantiated 1 time
complex_alu instantiated 1 time
PASS: resource check

iverilog: Icarus Verilog version 12.0 (stable) ()

=== tb_complex_addsub ===
tb_complex_addsub: Passed: 42371, Failed: 0
PASS: tb_complex_addsub

=== tb_complex_mul ===
tb_complex_mul: Passed: 22688, Failed: 0
PASS: tb_complex_mul

=== tb_complex_cpu ===
demo program: 8 instructions in 30 cycles
tb_complex_cpu: Passed: 1533, Failed: 0
PASS: tb_complex_cpu

=== tb_complex_cpu_spec ===
16 independent single-cycle instructions: 19 cycles
16 independent multiplies: 102 cycles
16 dependent multiplies: 117 cycles
cost of the interlock: 15 cycles over 15 dependent pairs
tb_complex_cpu_spec: Passed: 18, Failed: 0
PASS: tb_complex_cpu_spec

summary: 5/5 passed, 0 failed

```

جمع‌بندی

درس اصلی این آزمایش این است که یک قید منابع، معماری را عوض می‌کند نه فقط اندازه‌اش را. با اجازه‌ی داشتن چهار ضرب‌کننده، ضرب مختلط یک مدار ترکیبی یک‌کلاکی می‌شد و مرحله‌ی اجرا هم مثل بقیه‌ی مراحل پایپ‌لاین می‌شد. با یک ضرب‌کننده، همان عملیات به یک ماشین حالت چهارکلاکی تبدیل می‌شود و در نتیجه پایپ‌لاین به قفل و توقف نیاز پیدا می‌کند — چیزی که بدون آن قید اصلاً لازم نبود.

نکته‌ی دوم درباره‌ی تست است. چهار تست‌بنچ این آزمایش هیچ‌کدام نمی‌توانند نشان دهند که قید اصلی رعایت شده است، چون مداری با دو ضرب‌کننده همان جواب‌های درست را در زمان کمتری می‌دهد و همه‌ی آن‌ها را پاس می‌کند. برای ویژگی‌هایی که درباره‌ی ساختار مدارند و نه رفتار آن، باید سراغ خود متن رفت؛ به همین دلیل شمارش نمونه‌سازی‌ها به‌عنوان اولین تست به اسکرپیت اضافه شد.

خروجی سنتز (Quartus Prime)

طرح روی Intel Quartus Prime Lite Edition 21.1.0 با موجودیت سطح بالای complex_cpu سنتز و Place & Route شد. دستگاه هدف: Cyclone V (5CGXFC7C7F23C8).

مورد	نتیجه
وضعیت کامپایل	موفق (۰ خطا، ۱۹ هشدار)
Logic utilization (ALMs)	۶۹۹/۵۶۴۸۰ (۱٪)
Total registers	۱۲۷۲
Total pins	۹۰/۲۶۸ (۳۴٪)
Block memory bits	۰
DSP Blocks	۱/۱۵۶ (< ۱٪)

Quartus Prime Lite Edition - D:\university\DSd lab\complex_cpu_project\complex_cpu - complex_cpu

File Edit View Project Assignments Processing Tools Window Help

complex_cpu

Project Navigator Hierarchy

EntityInstance

Cyclone V: 5CGXFC7C7F23C8

complex_cpu

complex_alu

add_unitu_add

mul_unitu_mul

imemcu_imem

regfileu_rf

Tasks

Compilation

Task

Compile Design

Analysis & Synthesis

Fitter (Place & Route)

Assembler (Generate program)

Timing Analysis

EDA Netlist Writer

Table of Contents

Flow Summary

Flow Settings

Flow Non-Default Global

Flow Elapsed Time

Flow OS Summary

Flow Log

Analysis & Synthesis

Fitter

Flow Messages

Flow Suppressed Messages

Assembler

Timing Analyzer

Flow Summary

Flow Status: Successful - Sat Aug 29 21:21:40 2026

Quartus Prime Version: 21.1.0 Build 842 10/21/2021 SJ Lite Edition

Revision Name: complex_cpu

Top-level Entity Name: complex_cpu

Family: Cyclone V

Device: 5CGXFC7C7F23C8

Timing Models: Final

Logic utilization (in ALMs): 699 / 56,480 (1 %)

Total registers: 1272

Total pins: 90 / 268 (34 %)

Total virtual pins: 0

Total block memory bits: 0 / 7,024,640 (0 %)

Total DSP Blocks: 1 / 156 (< 1 %)

Total HSSI RX PCSs: 0 / 6 (0 %)

Total HSSI PMA RX Deserializers: 0 / 6 (0 %)

Total HSSI TX PCSs: 0 / 6 (0 %)

Total HSSI PMA TX Serializers: 0 / 6 (0 %)

Total PLLs: 0 / 13 (0 %)

Total DLLs: 0 / 4 (0 %)

IP Catalog

Installed IP

Project Directory

No Selection Available

Library

Basic Functions

DSP

Interface Protocols

Memory Interfaces and Controllers

Processors and Peripherals

University Program

Search for Partner IP

Messages

Type ID Message

332140 No Recovery paths to report

332140 No Removal paths to report

332146 worst-case minimum pulse width slack is -0.091

332114 Report Metastability: Found 32 synchronizer chains.

332102 Design is not fully constrained for setup requirements

332102 Design is not fully constrained for hold requirements

Quartus Prime Timing Analyzer was successful. 0 errors, 6 warnings

293000 Quartus Prime Full Compilation was successful. 0 errors, 19 warnings

System Processing (135)

100% 00:06:30